

| Document id | Title                     | Organisation /Author | Date          | Status |
|-------------|---------------------------|----------------------|---------------|--------|
|             | ifcOWL Final bSI Standard | LDWG                 | 29 March 2017 | FINAL  |

## **BuildingSMART Final Standard: The ifcOWL ontology**

### **Executive summary:**

This document outlines the ifcOWL ontology that is recommended and proposed by the BuildingSMART Linked Data Working Group (LDWG) for final standardisation within buildingSMART, alongside the already existing IFC standard in EXPRESS and XSD. This document lists:

1. Why have an ifcOWL ontology?
2. The EXPRESS-to-OWL conversion routine followed to obtain the ifcOWL ontology
3. Reference implementations
4. Validation results
5. A printout of the full resulting ifcOWL ontology
6. A printout of the full HTML documentation of the resulting ifcOWL ontology

Overall, this document shows the consensus-building processes, implementation processes, and validation processes that have been organised by the Linked Data Working Group over the past 3 years.

## Table of Contents

|                                                                                          |    |
|------------------------------------------------------------------------------------------|----|
| Table of Contents.....                                                                   | 2  |
| 1. Why have an ifcOWL ontology?.....                                                     | 3  |
| 2. EXPRESS-to-OWL conversion routine .....                                               | 5  |
| 2.1. Introduction to the EXPRESS-to-OWL conversion routine .....                         | 6  |
| 2.2. Schema definition.....                                                              | 6  |
| 2.3. Simple data type declarations .....                                                 | 8  |
| 2.4. Defined (named) data type declarations.....                                         | 9  |
| 2.5. Aggregation data type declarations .....                                            | 10 |
| 2.5.1 SET data type declarations.....                                                    | 10 |
| 2.5.2 LIST and ARRAY data type declarations .....                                        | 11 |
| 2.6. Constructed data type declarations .....                                            | 13 |
| 2.6.1 Enumeration data type declarations.....                                            | 13 |
| 2.6.2 Select data type declarations.....                                                 | 13 |
| 2.7. Entity (named) data type declarations .....                                         | 14 |
| 2.7.1 Regular attributes .....                                                           | 16 |
| 2.7.2 Aggregation attributes .....                                                       | 18 |
| 2.7.3 Optional attributes .....                                                          | 19 |
| 2.7.4 Inverse attributes.....                                                            | 20 |
| 2.7.5 Derive and where attributes.....                                                   | 21 |
| 2.8. FUNCTION declarations.....                                                          | 22 |
| 2.9. RULE declarations.....                                                              | 22 |
| 3. Reference implementations.....                                                        | 23 |
| 3.1 Open source JAVA EXPRESS-to-OWL converter.....                                       | 23 |
| 3.2 Closed source C++ EXPRESS-to-OWL converter .....                                     | 23 |
| 3.3 Open source C# EXPRESS-to-OWL converter .....                                        | 23 |
| 4. Validation results.....                                                               | 25 |
| 4.1. Tools used for converting IFC-SPF files to RDF files (validation purpose only)..... | 25 |
| 4.1.1. Open-source IFC-to-RDF JAVA project .....                                         | 26 |
| 4.1.2. The IfcDoc tool .....                                                             | 26 |
| 4.2. Tools used for converting RDF files to IFC-SPF files (validation purpose only)..... | 27 |
| 4.3. Validation results .....                                                            | 27 |
| 5. Pointer to the full resulting ifcOWL ontology.....                                    | 31 |
| 6. Pointer to the full HTML documentation of the resulting ifcOWL ontology.....          | 32 |

## 1. Why have an ifcOWL ontology?

The Industry Foundation Classes (IFC) are a well-known and agreed standard to capture data available and used throughout the international construction industry. IFC has always been available as an EXPRESS schema, and a bit more recently also as an XSD schema. These are in fact two formats to capture the same content and structure using a different computer-interpretable language. In this document, **we propose a third format, ifcOWL**, to capture again the same content and structure in yet another computer-interpretable language, namely the Web Ontology Language or OWL.

There is one key reason why we can use this third format. Namely, **it enables modern web and data technologies to better access and use the information contained in building models (IFC), well beyond the traditional construction industry** (i.e. technologies used in support of linked data, the semantic web, Internet of Things, Big Data).

ifcXML provides a standardised way to represent IFC models in XML and thus allows access to the model instance information using broadly available XML code libraries and improve the uptake and reusability of the contained model information. Like with ifcXML, the goal of ifcOWL is to provide a data representation (RDF and OWL), true to the original IFC schema, that is usable by modern and broadly standardised and adopted advanced web technologies that **support linking, querying and reasoning**. Hence, ifcOWL opens three major domains with a high number of use cases that are currently close to unavailable in construction industry:

1. **Direct linking** with other data available on the web in the same format (RDF, OWL):
  - a. GIS data
  - b. Product data
  - c. Sensor data
  - d. Control system data
  - e. IoT data
  - f. ...
2. Usage of logical **reasoning** engines, which enable the execution of logical IF-THEN rules in combination with available IFC data. This can be useful for:
  - a. Automated building compliance checking
  - b. Model-checking
  - c. Simple data transformations
  - d. Simplification of BIM model data
3. Usage of **query** languages and engines randomly available on the web, e.g. SPARQL. This can be useful for:
  - a. Easily retrieving data as an advanced end user
  - b. Generating small subsets of building data (as query results), which may be understood as micro Model View Definitions (microMVDs)

Note that the direct linking to other data sets allows to expand into other domains (as listed above), without having to continuously expand the domain of IFC itself. Hence, IFC can focus on its core domain, which is construction industry, while still allowing links outside that area.

We propose this ifcOWL ontology to be a **direct translation from the IFC EXPRESS schema to OWL**, which is the exact same procedure used to come to the XSD schema of IFC. This has some advantages:

1. Future-proof: The translation process can be applied to any subsequent release of IFC based on EXPRESS
2. Backward-compatible: any available tool working with regular IFC files remains to be fully supported in any workflow or toolchain.

There is also one potential key disadvantage: there is **no paradigm shift**. Namely, the IFC schema is not re-evaluated in itself, and a generic conversion pattern does not make many exceptions, so the resulting ifcOWL ontology does not follow the common best practices in the (semantic) web domain: e.g. re-use of existing ontologies, preference for small, modular ontologies. This feature is sacrificed here in favour of a clear alignment with the existing IFC standard.

Yet, the technology (RDF/OWL) is a language that is far better understood over the web than EXPRESS and STEP. Whereas EXPRESS and STEP reside primarily in the product modelling and construction industry (although also these industries shift rapidly towards the web), most other domain have shifted towards the usage of web technologies long time ago, including combinations of XML, JSON, RDF, and OWL to capture data. Making IFC available in such a language will provide **a reference placeholder for anyone outside the construction industry** for intensively using and tapping into that construction data.

This document reports the process used to develop, assess and validate the ifcOWL ontology in accordance with the aims set out above, as well as documenting the finalised mapping and results.

## 2. EXPRESS-to-OWL conversion routine

The best place to start building an ifcOWL ontology, is in the well-accepted Industry Foundation Classes (IFC), which consists mainly of an EXPRESS schema and a corresponding XSD schema. The EXPRESS schema and XSD schema form the standard, accompanied by the documentation provided via <http://www.buildingsmart-tech.org/specifications>. At the moment of writing, the XSD schema is generated from the EXPRESS schema (using the IfcDoc tool), making the EXPRESS schema the key computer-interpretable reference for the IFC standard.

The LDWG thus agrees that an ifcOWL standard should start from the EXPRESS schema in IFC, and that there is a need for a standardised EXPRESS-to-OWL conversion routine, from which the ifcOWL standard naturally flows.

As with any translation from one language to another, such a conversion routine can take many directions. This is clearly visible from the many publications on the topic from 2005 onwards. Hence, the LDWG has agreed on a number of criteria that need to be met by this routine. In order of importance, the following three criteria have been agreed on:

1. The ifcOWL ontology must be in OWL2 DL, so that it can be properly used by readily available reasoning and query engines.
2. The ifcOWL ontology must match the original EXPRESS schema as closely as possible.
3. The ifcOWL ontology primarily aims at supporting the conversion of IFC instance files (IFC-SPF) into equivalent RDF files. Thus, herein it is of secondary importance that an instance RDF file can be modelled from scratch using the ifcOWL ontology and an ontology editor.

The importance and impact of these three criteria cannot be overestimated. They shape the resulting ifcOWL ontology.

Even considering these three criteria, there are numerous ways to map an EXPRESS schema onto an OWL ontology. This has been investigated in depth and at length during many telephone conferences and live meetings of LDWG group members. This investigation has culminated in the Technical Session of the 3rd International Workshop on Linked Data in Architecture and Construction (LDAC), held in TU Eindhoven in July 2015. This workshop was attended by most, if not all, the people that represent the Linked Data Working Group. Moreover, it was also attended by the key people that have been investigating EXPRESS-to-OWL conversion routines since 2005, AND representative industrials.

The report at [http://ldac-2015.bwk.tue.nl/LDAC\\_2015\\_workshopreport.pdf](http://ldac-2015.bwk.tue.nl/LDAC_2015_workshopreport.pdf) lists the decisions made in this workshop. These decisions take into account the above three criteria. These decisions are followed in the current proposal. Many of the considerations in this Technical Report are also discussed at length in the article by P. Pauwels and W. Terkaj (2016) *EXPRESS to OWL for construction industry: towards a recommendable and usable ifcOWL ontology*. *Automation in Construction*, Volume 63,

March 2016, Pages 100–133, [doi:10.1016/j.autcon.2015.12.003](https://doi.org/10.1016/j.autcon.2015.12.003). This conversion routine has been presented to the buildingSMART Standardisation Committee on 12 Oct. 2015 (Ref. TR\_LDWG001) and has been presented in-detail during the Singapore International Standardisation Meeting (ISM) in October 2015. No comments have been received since that date, which implies agreement to the proposal. From that moment onwards, the LDWG has focused entirely on the implementation, use case and validation efforts starting from this agreed and accepted EXPRESS-to-OWL conversion routine.

## 2.1. Introduction to the EXPRESS-to-OWL conversion routine

The EXPRESS-to-OWL conversion presented in this document is relying on the IFC4\_ADD1.exp EXPRESS schema. All examples used in this document come from this schema. The conversion procedure is also applicable on IFC2X3\_Final.exp, IFC2X3\_TC1.exp, IFC4.exp, and IFC4\_ADD2.exp.

Examples are documented using Fragments of EXPRESS declarations and Fragments of OWL declarations in the Turtle syntax (TTL).

## 2.2. Schema definition

An EXPRESS schema contains exactly one SCHEMA declaration that covers the entire file, thereby assigning the body of this file to the particular schema declaration. An EXPRESS schema may import definitions from other schemas using the REFERENCE FROM keywords. The IFC schema is self-contained and there is no import of other schemas (see Fragment 1).

```
SCHEMA IFC4;  
...  
END_SCHEMA;
```

**Fragment 1 - Printout of the SCHEMA declaration present in IFC4 ADD1.exp.**

Each EXPRESS schema is converted in a separate ontology, each identified by a unique URI. The following URI design is used for these ontologies: [http://www.buildingsmart-tech.org/ifcOWL/\[schemaName\]](http://www.buildingsmart-tech.org/ifcOWL/[schemaName]), with [schemaName] being one of the available IFC schema names (IFC2X3\_Final, IFC2X3\_TC1, IFC4, IFC4\_ADD1, IFC4\_ADD2).

Fragment 2 shows in what this results in OWL. A number of Dublin Core (dce) metadata annotations is added to the ontology declaration, indicating details about the origins of the OWL ontology. The vann metadata annotations indicate preferred namespace URI and namespace prefix, and the cc:license property indicates which license is associated to the ontology.

```
<http://www.buildingsmart-tech.org/ifcOWL/IFC4_ADD1>
rdf:type owl:Ontology ;
dce:creator    "Pieter Pauwels (pipauwel.pauwels@ugent.be)" ,
               "Walter Terkaj (walter.terkaj@itia.cnr.it)" ;
dce:contributor    "Jakob Beetz (j.beetz@tue.nl)" ,
                   "Maria Poveda Villalon (mpoveda@fi.upm.es)" ,
                   "Nam Vu Hoang (nam.vuhoang@gmail.com )" ,
                   "Seppo Törmä (seppo.torma@aalto.fi)" ,
                   "Jyrki Oraskari (jyrki.oraskari@aalto.fi)";
rdfs:comment    "Ontology automatically generated from the EXPRESS
                schema IFC4_ADD1' using the 'IFC-to-RDF' converter developed by Pieter
                Pauwels (pipauwel.pauwels@ugent.be), based on the earlier versions from
                Jyrki Oraskari (jyrki.oraskari@aalto.fi) and Davy Van Deursen (davy.van-
                deursen@ugent.be)" ;
dce:title       "IFC4_ADD1" ;
dce:description "OWL ontology for the IFC conceptual data schema and ex-
                change file format for Building Information Model (BIM)
                data" ;
dce:date        "2016/01/22" ;
dce:identifier  "IFC4_ADD1" ;
dce:language    "en" ;
vann:preferredNamespacePrefix "ifc" ;
vann:preferredNamespaceUri    "http://www.buildingsmart-
                               tech.org/ifcOWL/IFC4_ADD1" ;
owl:imports      <https://w3id.org/express> ;
cc:license       <http://creativecommons.org/licenses/by/3.0/> ;
```

**Fragment 2 - Definition of the ifcOWL ontology.**

The ontologies rely on a number of existing ontologies. Fragment 3 shows the prefixes and the URIs of the ontologies that are relied upon.

A key imported ontology is the EXPRESS ontology, which is imported with namespace <https://w3id.org/express#> and prefix 'expr:'. This ontology contains the classes and properties that are specific to the EXPRESS language and are not actually part of the IFC schema (e.g. expr:hasDouble). This EXPRESS ontology in turn imports a LIST ontology, which is accessible via <https://w3id.org/list>. This ontology is heavily used in the ifcOWL ontology to represent any list in IFC, as will be documented in the remainder of the current document (e.g. list:hasNext).

```
@base <http://www.buildingsmart-tech.org/ifcOWL/IFC4_ADD1 .
@prefix : <http://www.buildingsmart-tech.org/ifcOWL/IFC4_ADD1#> .
@prefix ifc: <http://www.buildingsmart-tech.org/ifcOWL/IFC4_ADD1#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix dce: <http://purl.org/dc/elements/1.1/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix cc: <http://creativecommons.org/ns#> .
@prefix expr: <https://w3id.org/express#> .
@prefix vann: <http://purl.org/vocab/vann/> .
```

**Fragment 3 - Printout of the ontologies that are used by the proposed ifcOWL ontology.**

## 2.3. Simple data type declarations

Simple data types (i.e. NUMBER, REAL, INTEGER, LOGICAL, BOOLEAN, STRING, BINARY) are commonly used in an IFC schema. As these data types are inherently part of the EXPRESS language, rather than the IFC schema, these data types are maintained in a separate ontology, namely the EXPRESS ontology prefixed as 'expr:' in Fragment 3. This ontology is out of scope for the currently proposed ifcOWL standard, but we list its key features here anyway. In the EXPRESS ontology, simple data types (expr:REAL in Fragment 4) are available as classes with datatype properties (expr:hasDouble in Fragment 4) pointing to xsd literals (xsd:double in Fragment 4).

```
expr:REAL
  rdf:type owl:Class ;
  rdfs:subClassOf
  [
    rdf:type owl:Restriction ;
    owl:allValuesFrom xsd:double ;
    owl:onProperty expr:hasDouble
  ] .

expr:hasDouble
  rdf:type owl:DatatypeProperty ;
  rdf:type owl:FunctionalProperty ;
  rdfs:label "hasDouble" ;
  rdfs:domain
  [
    rdf:type owl:Class ;
    owl:unionOf ( expr:NUMBER expr:REAL )
  ] ;
  rdfs:range xsd:double .
```

**Fragment 4 - Printout of the RDF graph representation for the simple data type REAL in the ifcOWL schema.**



The LOGICAL data type is a special case, as there is no equivalent XSD literal datatype which can hold the values TRUE, FALSE, and UNKNOWN. For this particular datatype, a special `expr:LogicalEnum` class is constructed that is an enumeration of the individuals `expr:TRUE`, `expr:FALSE`, and `expr:UNKNOWN` (see Fragment 5). The rest of the conversion pattern follows the same approach as for the other simple data types, namely with a class `expr:LOGICAL` and an object property `expr:hasLogical` pointing to one of the three enumeration values.

```
expr:hasLogical a owl:FunctionalProperty , owl:ObjectProperty ;
  rdfs:domain expr:LOGICAL ;
  rdfs:label "hasLogical" ;
  rdfs:range expr:LogicalEnum .

expr:LOGICAL a owl:Class ;
  rdfs:subClassOf [ a owl:Restriction ;
    owl:allValuesFrom expr:LogicalEnum ;
    owl:onProperty expr:hasLogical
  ] .

expr:LogicalEnum a owl:Class ;
  rdfs:subClassOf expr:ENUMERATION .

expr:TRUE a expr:LogicalEnum , owl:NamedIndividual ;
  rdfs:label "TRUE" .

expr:FALSE a expr:LogicalEnum , owl:NamedIndividual ;
  rdfs:label "FALSE" .

expr:UNKNOWN a expr:LogicalEnum , owl:NamedIndividual ;
  rdfs:label "UNKNOWN" .
```

**Fragment 5 - Printout of the RDF graph representation of the `expr:LOGICAL` class and its three named individuals.**

Also the classes and properties in Fragment 5 are specific to EXPRESS and are thus part of a separate EXPRESS ontology at <https://w3id.org/express>.

## 2.4. Defined (named) data type declarations

A number of defined data types is typically available in any EXPRESS schema of IFC. Most declarations are similar to the one given in Fragment 6 (`IfcAreaDensityMeasure`), where a type name is given (e.g. `IfcAreaDensityMeasure`) together with a reference to a simple data type (e.g. `REAL`).

```
TYPE IfcAreaDensityMeasure = REAL;  
END_TYPE;
```

**Fragment 6 - Printout of the defined data type declaration IfcAreaDensityMeasure.**

These defined data types are converted into classes, which are then declared as subclasses of the data type to which they are referring (see Fragment 7). Note that the actual data types (e.g. expr:REAL) are again part of the EXPRESS ontology that is outside the scope of the current standard.

```
ifc:IfcAreaDensityMeasure  
  rdf:type owl:Class ;  
  rdfs:subClassOf expr:REAL .
```

**Fragment 7 - Printout of the RDF graph representation for the IfcAreaDensityMeasure defined data type.**

## 2.5. Aggregation data type declarations

Often used data types in the IFC schema are the aggregation data types SET, LIST, and ARRAY. There exists also a BAG data type in EXPRESS, but it is not used in the IFC schema at the moment, so it is also not handled in the conversion routine proposed here.

### 2.5.1 SET data type declarations

The SET aggregation data types are unordered aggregations of instances that are supposed to be different from each other. These aggregations are typically declared for attributes of EXPRESS entities (see Fragment 8).

```
ENTITY IfcArbitraryProfileDefWithVoids  
...  
  InnerCurves : SET [1:?] OF IfcCurve;  
...  
END_ENTITY;
```

**Fragment 8 - Printout of a non-OPTIONAL SET data type declaration in RDF, with a lower cardinality restriction of 1 (at least one instance should be present in the SET).**

An unordered aggregation is naturally represented in OWL through a common non-functional object property that can be assigned an unlimited number of times to the same instance (see top of Fragment 9). OWL restrictions for this non-functional property allow to represent object type and cardinality constraints (see bottom of Fragment 9).

```

ifc:innerCurves_IfcArbitraryProfileDefWithVoids
    a      owl:ObjectProperty ;
    rdfs:domain ifc:IfcArbitraryProfileDefWithVoids ;
    rdfs:label "InnerCurves" ;
    rdfs:range ifc:IfcCurve .

ifc:IfcArbitraryProfileDefWithVoids
    rdfs:subClassOf
        [
            rdf:type owl:Restriction ;
            owl:minQualifiedCardinality "1"^^xsd:nonNegativeInteger ;
            owl:onClass ifc:IfcCurve ;
            owl:onProperty ifc:innerCurves_IfcArbitraryProfileDefWithVoids
        ] ;
    rdfs:subClassOf
        [
            rdf:type owl:Restriction ;
            owl:allValuesFrom ifc:IfcCurve ;
            owl:onProperty ifc:innerCurves_IfcArbitraryProfileDefWithVoids
        ] .

```

**Fragment 9 – Printout of the non-functional InnerCurves object property, and of the owl:Class declaration for the IfcArbitraryProfileDefWithVoids entity data type declaration, including restrictions.**

### 2.5.2 LIST and ARRAY data type declarations

LIST and ARRAY data type declarations are harder to express in OWL declarations. They each represent an ordered collection of elements. In the proposed conversion routine, we propose to use the same conversion routine for both data type declarations. An example LIST data type declaration is given in Fragment 10, showing a LIST of at least 3 and at most 4 INTEGER instances.

```

TYPE IfcCompoundPlaneAngleMeasure = LIST [3:4] OF INTEGER;
...
END_TYPE;

```

**Fragment 10 – An example LIST data type declaration.**

The LIST data type declaration in Fragment 10 is declared as part of a defined data type declaration (see Fragment 6 and 7). Hence, the conversion routine for defined data type declarations is proposed for the IfcCompoundPlaneAngleMeasure data type, subsuming the expr:INTEGER\_List class (see Fragment 11).

```

ifc:IfcCompoundPlaneAngleMeasure
    rdf:type      owl:Class ;
    rdfs:subClassOf ifc:IfcDerivedMeasureValue , expr:INTEGER_List ;

```

**Fragment 11 – Printout of the RDF graph for the IfcCompoundPlaneAngleMeasure data type.**

The `expr:INTEGER_List` class is part of the EXPRESS ontology, again because this is actually part of the EXPRESS language rather than the IFC schema. In the EXPRESS ontology, it is defined as a subclass of the `list:OWList` class. This list ontology (`list:`) provides a number of object properties (e.g. `list:hasNext`) that are used in the `ifcOWL` ontology to represent the cardinality restrictions for the `LIST` data type. In the case of `IfcCompoundPlaneAngleMeasure (LIST [3:4])`, this results in the OWL restrictions shown in Fragment 12 and 13.

```
ifc:IfcCompoundPlaneAngleMeasure
rdfs:subClassOf [
  a owl:Restriction ;
  owl:onProperty list:hasNext ;
  owl:someValuesFrom [
    a owl:Restriction ;
    owl:onProperty list:hasNext ;
    owl:someValuesFrom [
      a owl:Restriction ;
      owl:onProperty list:hasNext ;
      owl:someValuesFrom expr:INTEGER_List
    ]
  ]
];
```

**Fragment 12 – Concatenation of OWL restrictions to indicate that there should be at least three instances in the `expr:INTEGER_List`.**

```
ifc:IfcCompoundPlaneAngleMeasure
rdfs:subClassOf [
  a owl:Restriction ;
  owl:allValuesFrom [
    a owl:Restriction ;
    owl:allValuesFrom [
      a owl:Restriction ;
      owl:allValuesFrom [
        a owl:Restriction ;
        owl:onClass expr:INTEGER_EmptyList ;
        owl:onProperty list:hasNext ;
        owl:qualifiedCardinality "1"^^xsd:nonNegativeInteger
      ];
      owl:onProperty list:hasNext
    ];
    owl:onProperty list:hasNext
  ];
  owl:onProperty list:hasNext
];
```

**Fragment 13 – Concatenation of OWL restrictions to indicate that there should be at most four instances in the `expr:INTEGER_List`.**

## 2.6. Constructed data type declarations

### 2.6.1 Enumeration data type declarations

Enumeration data type declarations are declared as presented in Fragment 14. The list of values given in this declaration limits the values that an instance of this enumeration data type can have.

```
TYPE IfcAddressTypeEnum = ENUMERATION OF
  (OFFICE
  ,SITE
  ,HOME
  ,DISTRIBUTIONPOINT
  ,USERDEFINED);
END_TYPE;
```

**Fragment 14 – Printout of the ENUMERATION data type declaration IfcAddressTypeEnum, which refers to a list of the allowed values within this enumeration (OFFICE, SITE, HOME, DISTRIBUTIONPOINT and USERDEFINED).**

In the proposed EXPRESS-to-OWL conversion routine, Enumeration data types are converted into regular OWL classes. The allowed values that are declared in EXPRESS, are also declared in ifcOWL as instances (i.e. Named Individuals) of the Enumeration class (Fragment 15). As can be seen in Fragment 15, a Named Individual can be an instance of a number of OWL enumeration classes (e.g. ifc:SITE).

```
ifc:IfcAddressTypeEnum
  a          owl:Class ;
  rdfs:subClassOf    expr:ENUMERATION .

ifc:DISTRIBUTIONPOINT
  a          ifc:IfcAddressTypeEnum , owl:NamedIndividual ;
  rdfs:label  "DISTRIBUTIONPOINT" .

ifc:SITE
  a          ifc:IfcAddressTypeEnum , ifc:IfcCrewResourceTypeEnum ,
            ifc:IfcAssemblyPlaceEnum , owl:NamedIndividual ;
  rdfs:label  "SITE" .
```

**Fragment 15 – Printout of the RDF graph for the IfcAddressTypeEnum class, also showing two Named Individuals of this class (ifc:DISTRIBUTIONPOINT and ifc:SITE).**

### 2.6.2 Select data type declarations

A select data type declaration is identified by the keyword SELECT, as shown in Fragment 16. Any instantiation of such data type must be an instance of one of the listed types or entities.

```
TYPE IfcMetricValueSelect = SELECT
  (IfcAppliedValue
  ,IfcMeasureWithUnit
  ,IfcReference
  ,IfcTable
  ,IfcTimeSeries
  ,IfcValue);
END_TYPE;
```

**Fragment 16 – Printout of the SELECT data type declaration IfcMetricValueSelect, which refers to a list of allowed data type instantiations (IfcAppliedValue, IfcMeasureWithUnit, IfcReference, IfcTable, IfcTimeSeries, and IfcValue).**

Similar to the conversion of Enumeration data types, Select data types are converted into OWL classes. The classes that appear in the list of data types in the Select data type declaration are declared as subclasses of the Select data type. So, for the example in Fragment 16, ifc:IfcValue is declared as a subtype of ifc:IfcMetricValueSelect. Also in this case, multiple superclasses can be available.

```
ifc:IfcMetricValueSelect
  a          owl:Class ;
  rdfs:subClassOf  expr:SELECT .

ifc:IfcValue
  a          owl:Class ;
  rdfs:subClassOf  expr:SELECT , ifc:IfcAppliedValueSelect , ifc:IfcMetricValueSe-
lect .
```

**Fragment 17 – Printout of the RDF graph for the IfcMetricValueSelect class and its subclass: IfcValue.**

## 2.7. Entity (named) data type declarations

The core of the IFC EXPRESS schemas is represented in the Entity data type declarations. Two example Entity data type declarations are given in Fragment 18 and 19 (IfcBSplineCurve and IfcObject). These two examples contain most, if not all, attribute declaration types that can be part of an entity data type declaration. So they serve as perfect examples for documenting the EXPRESS-to-OWL conversion routine in the current document.

```

ENTITY IfcBSplineCurve
  ABSTRACT SUPERTYPE OF (ONEOF (IfcBSplineCurveWithKnots))
  SUBTYPE OF (IfcBoundedCurve);
    Degree : IfcInteger;
    ControlPointsList : LIST [2:?] OF IfcCartesianPoint;
    CurveForm : IfcBSplineCurveForm;
    ClosedCurve : IfcLogical;
    SelfIntersect : IfcLogical;
  DERIVE
    UpperIndexOnControlPoints : IfcInteger := (SIZEOF(ControlPointsList) - 1);
    ControlPoints : ARRAY [0:UpperIndexOnControlPoints] OF IfcCartesianPoint
:=
    IfcListToArray(ControlPointsList,0,UpperIndexOnControlPoints);
  WHERE
    SameDim : SIZEOF(QUERY(Temp <* ControlPointsList | Temp.Dim <>
    ControlPointsList[1].Dim)) = 0;
END_ENTITY;

```

**Fragment 18 – Printout of the IfcBSplineCurve Entity data type declaration.**

```

ENTITY IfcObject
  ABSTRACT SUPERTYPE OF (ONEOF(IfcActor,IfcControl,IfcGroup,IfcProcess,IfcProduct,IfcResource))
  SUBTYPE OF (IfcObjectDefinition);
    ObjectType : OPTIONAL IfcLabel;
  INVERSE
    IsDeclaredBy : SET [0:1] OF IfcRelDefinesByObject FOR RelatedObjects;
    Declares : SET [0:?] OF IfcRelDefinesByObject FOR RelatingObject;
    IsTypedBy : SET [0:1] OF IfcRelDefinesByType FOR RelatedObjects;
    IsDefinedBy : SET [0:?] OF IfcRelDefinesByProperties FOR RelatedObjects;
  WHERE
    UniquePropertySetNames : IfcUniqueDefinitionNames(IsDefinedBy);
END_ENTITY;

```

**Fragment 19 – Printout of the IfcObject Entity data type declaration.**

All entity data types are commonly converted into OWL class declarations, as displayed in Fragment 20 for IfcBSplineCurve and IfcObject. A subclass relation (using `rdfs:subClassOf`) is defined whenever there is an EXPRESS declaration `SUBTYPE OF()`.

If an entity data type is an `ABSTRACT SUPERTYPE OF()`, then the corresponding OWL class is defined as the subclass of the union (`owl:unionOf`) of its subclasses. For instance, IfcObject is the subclass of the union of IfcActor, IfcControl, IfcGroup, IfcProcess, IfcProduct, IfcResource, as shown in Fragment 20.

In addition, if the keyword `ONE OF` is used after `SUPERTYPE OF`, then it means that the subclasses must be also disjoint; this is taken into account by explicitly declaring the disjointness (`owl:disjointWith`).

For instance, `IfcBSplineCurve` is a SUBTYPE of `IfcBoundedCurve` that is in turned declared as “ABSTRACT SUPERTYPE OF (ONEOF(`IfcBSplineCurve`, `IfcCompositeCurve`, `IfcIndexedPolyCurve`, `IfcPolyline`, `IfcTrimmedCurve`))”. Therefore, `IfcBSplineCurve` must be disjoint with `IfcCompositeCurve`, `IfcIndexedPolyCurve`, `IfcPolyline`, `IfcTrimmedCurve`, as shown in Fragment 20.

```
ifc:IfcBSplineCurve
  rdf:type owl:Class ;
  rdfs:subClassOf ifc:IfcBoundedCurve ;
  rdfs:subClassOf
    [
      rdf:type owl:Class ;
      owl:unionOf
        (
          ifc:IfcBSplineCurveWithKnots
        )
    ] ;
  owl:disjointWith
    ifc:IfcPolyline,
    ifc:IfcIndexedPolyCurve,
    ifc:IfcCompositeCurve,
    ifc:IfcTrimmedCurve .

ifc:IfcObject rdf:type owl:Class ;
  rdfs:subClassOf ifc:IfcObjectDefinition ,
    [
      rdf:type owl:Class ;
      owl:unionOf
        (
          ifc:IfcActor
          ifc:IfcControl
          ifc:IfcGroup
          ifc:IfcProcess
          ifc:IfcProduct
          ifc:IfcResource
        )
    ] ;
  owl:disjointWith ifc:IfcTypeObject .
```

**Fragment 20 – Printout of the RDF graph representation for the `IfcBSplineCurve` and `IfcObject` class.**

### 2.7.1 Regular attributes

For each entity data type, a number of regular attributes is typically declared. An example of such a regular attribute declaration is given in Fragment 21. In this case, an attribute Degree is declared that points from an `IfcBSplineCurve` instance to an `IfcInteger` instance.



```
ENTITY IfcBSplineCurve
...
Degree : IfcInteger;
...
END_ENTITY;
```

**Fragment 21 – Printout of a regular attribute declaration in EXPRESS.**

Such regular attributes are commonly converted into OWL object properties (see owl:ObjectProperty in Fragment 22). As the Degree attribute is not pointing to a SET data type declaration, there is only 1 Degree attribute allowed, which is taken into account by declaring the object property as a functional object property (owl:FunctionalProperty).

In the proposed EXPRESS-to-OWL conversion routine, each object property is declared with exactly one data type in its range and exactly one data type in its domain (see Fragment 22), mainly because attributes in EXPRESS always point to at most one EXPRESS data type, and they are defined only within the scope of the entity data type declaration in which they appear. To guarantee the uniqueness of the object properties, their local URI is named following the naming convention [nameOfAttribute]\_of\_[NameOfEntity] (see Fragment 22). It must be noted that the object property local URI always starts with a lower case letter to be compliant with the [Ontology Best Practices](#). The original name of the attribute is kept as the label (rdfs:label) of the object property.

```
ifc:degree_IfcBSplineCurve
a          owl:ObjectProperty , owl:FunctionalProperty ;
rdfs:domain ifc:IfcBSplineCurve ;
rdfs:label  "Degree" ;
rdfs:range  ifc:IfcInteger .
```

**Fragment 22 – Printout of the OWL object property declaration for a regular EXPRESS attribute.**

In addition to the object property declarations as in Fragment 22, an object type restriction and a cardinality restriction are also added to the OWL class declaration, as is displayed in Fragment 23. In this example, the cardinality restriction indicates that there should always be exactly one instantiation of this property, as the EXPRESS attribute is not OPTIONAL and also does not point to a SET aggregation data type.

```
ifc:IfcBSplineCurve
  rdfs:subClassOf
    [
      rdf:type owl:Restriction ;
      owl:allValuesFrom ifc:IfcInteger ;
      owl:onProperty ifc:degree_IfcBSplineCurve
    ] ;
  rdfs:subClassOf
    [
      rdf:type owl:Restriction ;
      owl:qualifiedCardinality "1"^^xsd:nonNegativeInteger ;
      owl:onProperty ifc:degree_IfcBSplineCurve ;
      owl:onClass ifc:IfcInteger
    ] .
```

**Fragment 23 – Printout of the restrictions that are added to the IfcBSplineCurve class to handle object type and cardinality restrictions.**

### 2.7.2 Aggregation attributes

A number of attributes points to aggregation data types, in the case of IFC limited to LIST, SET, and ARRAY data types. Fragments 8 to 13 already presented the way in which these aggregation data types are handled. This is repeated in Fragment 24 to 26.

```
ENTITY IfcBSplineCurve
...
  ControlPointsList : LIST [2:?] OF IfcCartesianPoint;
...
END_ENTITY;
```

**Fragment 24 – Printout of the EXPRESS declaration of an attribute pointing to a LIST aggregation data type.**

```
ifc:controlPointsList_IfcBSplineCurve
  rdfs:label      "ControlPointsList" ;
  rdfs:domain     ifc:IfcBSplineCurve ;
  rdfs:range      ifc:IfcCartesianPoint_List ;
  rdf:type        owl:FunctionalProperty, owl:ObjectProperty .
```

**Fragment 25 – Printout of the OWL object property declaration for an EXPRESS attribute that points to an aggregation data type.**

```

ifc:IfcBSplineCurve
  rdfs:subClassOf [
    a
    owl:allValuesFrom ifc:IfcCartesianPoint_List ;
    owl:onProperty ifc:controlPointsList_IfcBSplineCurve
  ] ;
  rdfs:subClassOf [
    a
    owl:onClass ifc:IfcCartesianPoint_List ;
    owl:onProperty ifc:controlPointsList_IfcBSplineCurve ;
    owl:qualifiedCardinality "1"^^xsd:nonNegativeInteger
  ] ;
  rdfs:subClassOf [
    a owl:Restriction ;
    owl:allValuesFrom [
      a owl:Restriction ;
      owl:onProperty list:hasNext ;
      owl:someValuesFrom [
        a owl:Restriction ;
        owl:onProperty list:hasNext ;
        owl:someValuesFrom ifc:IfcCartesianPoint_List
      ]
    ]
  ] ;
  owl:onProperty ifc:controlPointsList_IfcBSplineCurve ] .

```

**Fragment 26 – Printout of the restrictions that are added to the IfcBSplineCurve class to handle object type and cardinality restrictions for the object property pointing to a LIST class.**

### 2.7.3 Optional attributes

The EXPRESS language provides the option to declare OPTIONAL attributes, as shown in Fragment 27. These attribute declarations are converted as if they were regular attributes (see Fragment 28), the only difference being that a maxQualifiedCardinality restriction is used (bottom of Fragment 29) instead of an exact qualifiedCardinality restriction (see Fragment 23).

```

ENTITY IfcObject
...
  ObjectType : OPTIONAL IfcLabel;
...
END_ENTITY;

```

**Fragment 27 – Printout of a regular attribute declaration in EXPRESS.**

```
ifc:objectType_IfcObject
  rdfs:label      "ObjectType" ;
  rdfs:domain     ifc:IfcObject ;
  rdfs:range      ifc:IfcLabel ;
  rdf:type        owl:FunctionalProperty, owl:ObjectProperty .
```

**Fragment 28 – Printout of the OWL object property declaration for an OPTIONAL EXPRESS attribute.**

```
ifc:IfcObject
  rdf:type owl:Class ;
  rdfs:subClassOf
    [
      rdf:type owl:Restriction ;
      owl:allValuesFrom ifc:IfcLabel ;
      owl:onProperty ifc:objectType_IfcObject
    ] ;
  rdfs:subClassOf
    [
      rdf:type owl:Restriction ;
      owl:maxQualifiedCardinality "1"^^xsd:nonNegativeInteger ;
      owl:onProperty ifc:objectType_IfcObject ;
      owl:onClass ifc:IfcLabel
    ] .
```

**Fragment 29 – Printout of the restrictions that are added to the IfcObject class to handle object type and cardinality restrictions for the OPTIONAL object property.**

## 2.7.4 Inverse attributes

The EXPRESS language allows to declare INVERSE attributes as well. Any such INVERSE statement declares an attribute that is inverse to an attribute that has been declared elsewhere and in the opposite direction. In the case of the IsDeclaredBy attribute in Fragment 30, for example, the attribute going in the inverse direction can be found in Fragment 31, namely the RelatedObjects attribute of the IfcRelDefinesByObject entity data type.

```
ENTITY IfcObject
  ...
  INVERSE IsDeclaredBy : SET [0:1] OF IfcRelDefinesByObject FOR RelatedObjects;
  ...
END_ENTITY;
```

**Fragment 30 – Printout of the INVERSE attribute IsDeclaredBy for the IfcObject entity (named) data type declaration.**

```
ENTITY IfcRelDefinesByObject
  SUBTYPE OF (IfcRelDefines);
  RelatedObjects : SET [1:?] OF IfcObject;
  RelatingObject : IfcObject;
END_ENTITY;
```

**Fragment 31 – Printout of the entity (named) data type declaration IfcRelDefinesByObject.**

The INVERSE attribute is converted using the regular conversion procedure, only adding an owl:inverseOf declaration.

```
ifc:isDeclaredBy_IfcObject
  rdfs:domain      ifc:IfcObject ;
  rdfs:range       ifc:IfcRelDefinesByObject ;
  owl:inverseOf  ifc:relatedObjects_IfcRelDefinesByObject ;
  rdf:type         owl:FunctionalProperty, owl:ObjectProperty .
```

**Fragment 32 – Printout of the OWL object property declaration for an INVERSE EXPRESS attribute.**

In two particular cases, an INVERSE attribute must not be converted in the ifcOWL ontology in order to keep the ontology consistent:

1. when an attribute has two or more INVERSE attributes, otherwise those inverse attributes would be considered equivalent by any OWL inference engine.
2. when a regular attribute or its INVERSE has a LIST or ARRAY as its range, otherwise there would be a domain/range mismatch and thus an inconsistent ontology.

## 2.7.5 Derive and where attributes

An example DERIVE attribute declaration is given in Fragment 33; an example WHERE attribute is given in Fragment 34. DERIVE and WHERE attributes are not handled in the current EXPRESS-to-OWL conversion routine.

```
ENTITY IfcBSplineCurve
  ABSTRACT SUPERTYPE OF (ONEOF(IfcBSplineCurveWithKnots))
  SUBTYPE OF (IfcBoundedCurve);
  ...
  DERIVE
    UpperIndexOnControlPoints : IfcInteger := (SIZEOF(ControlPointsList) - 1);
    ControlPoints : ARRAY [0:UpperIndexOnControlPoints] OF IfcCartesianPoint
      := IfcListToArray(ControlPointsList,0,UpperIndexOnControlPoints);
  ...
END_ENTITY;
```

**Fragment 33 – Printout of a DERIVE attribute in EXPRESS.**

```
TYPE IfcBoxAlignment = IfcLabel;  
...  
WHERE  
    WR1 : SELF IN ['top-left', 'top-middle', 'top-right', 'middle-left', 'center', 'middle-right', 'bottom-left', 'bottom-middle', 'bottom-right'];  
...  
END_TYPE;
```

**Fragment 34 – Printout of a WHERE attribute in EXPRESS.**

## 2.8. FUNCTION declarations

An example FUNCTION declaration is given in Fig. 35. The conversion of FUNCTION declarations is out of scope for the current proposed EXPRESS-to-OWL conversion routine.

```
FUNCTION IfcUniqueQuantityNames (Properties : SET [1:?] OF IfcPhysicalQuantity) :LOGICAL;  
  
LOCAL  
    Names : SET OF IfcLabel := [];  
END_LOCAL;  
  
REPEAT i:=1 TO HIINDEX(Properties);  
    Names := Names + Properties[i].Name;  
END_REPEAT;  
  
RETURN (SIZEOF(Names) = SIZEOF(Properties));  
END_FUNCTION;
```

**Fragment 35 – Printout of a FUNCTION declaration in EXPRESS.**

## 2.9. RULE declarations

An example RULE declaration is given in Fig. 36. The conversion of RULE declarations is out of scope for the current proposed EXPRESS-to-OWL conversion routine.

```
RULE IfcSingleProjectInstance  
    FOR (IfcProject);  
    WHERE  
        WR1 : SIZEOF(IfcProject) <= 1;  
END_RULE;
```

**Fragment 36 – Printout of a RULE declaration in EXPRESS.**

### 3. Reference implementations

The EXPRESS-to-OWL conversion routine has been implemented in a number of independent tools. These tools convert an available EXPRESS schema to a corresponding OWL ontology. These tools have been implemented by different people and using different programming languages. The resulting ontologies have been compared and discussed until they were all identical (iteration in setting up the above documented EXPRESS-to-OWL conversion routine).

#### 3.1 Open source JAVA EXPRESS-to-OWL converter

**URL:** <https://github.com/pipauwel/EXPRESStoOWL>

**Authors:** Pieter Pauwels, Jyrki Oraskari, Davy Van Deursen, Lewis McGibbney

**Description:** The EXPRESStoOWL project is a set of reusable Java components that allows to parse EXPRESS files and convert them into OWL ontologies in the context of the Industry Foundation Classes (IFC).

By executing this code with any of the available EXPRESS schemas (a .EXP file), a corresponding OWL ontology is obtained (a .TTL file).

The code relies on a JAVA software library that allows to parse EXPRESS and IFC-SPF files, namely the ifcParserLib (<https://github.com/pipauwel/ifcParserLib>).

The project is implemented as a MAVEN project.

#### 3.2 Closed source C++ EXPRESS-to-OWL converter

**Authors:** Walter Terkaj

**URL:** <http://www.terkaj.com/tools.html>

**Description:** C++ tool to automatically convert an EXPRESS schema into an OWL ontology. The tool was mainly applied to generate the ifcOWL ontology (Linked Data Working Group), but it can be used to convert any EXPRESS schema. In case of ifcOWL, the tool includes the option to convert also a subset of WHERE rules.

- download ExpressToOwl as zip file:  
<http://www.terkaj.com/tools/ExpressToOwl/ExpressToOwl.zip>
- readme:  
<http://www.terkaj.com/tools/ExpressToOwl/readme.txt>

More details can be found in readme file and related publications:

- Terkaj W, Sojic A (2015) Ontology-based Representation of IFC EXPRESS rules: an enhancement of the ifcOWL ontology. Automation in Construction, 57:188-201. [doi:10.1016/j.autcon.2015.04.010](https://doi.org/10.1016/j.autcon.2015.04.010)
- Pauwels P, Terkaj W (2016) EXPRESS to OWL for construction industry: Towards a recommendable and usable ifcOWL ontology. Automation in Construction, 63:100–133. [doi:10.1016/j.autcon.2015.12.003](https://doi.org/10.1016/j.autcon.2015.12.003)

#### 3.3 Open source C# EXPRESS-to-OWL converter

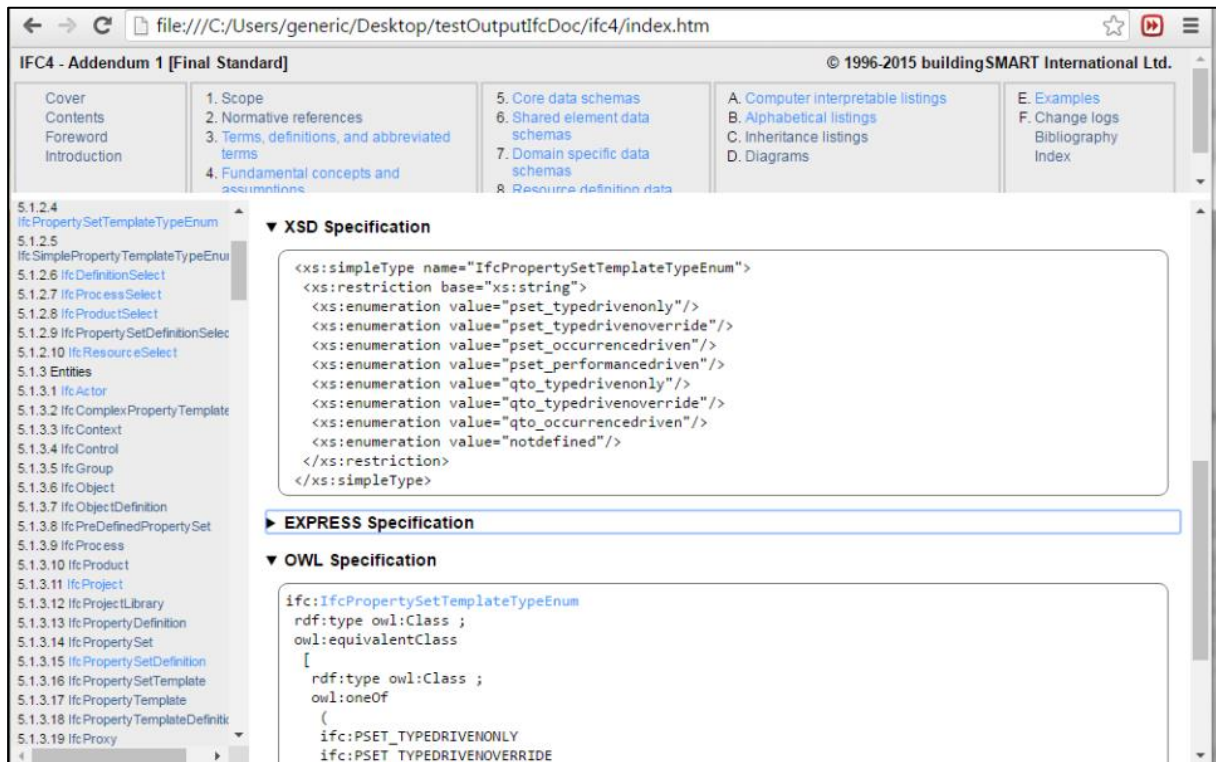
**Authors:** Pieter Pauwels, Walter Terkaj, Tim Chipman

**URL:** <https://github.com/BuildingSMART/IfcDoc>

## Description:

The IfcDoc tool, available for download and usage through <http://www.buildingsmart-tech.org/specifications/specification-tools/ifcdoc-tool>, contains the required functionality to make any EXPRESS information that the tool contains also available as OWL information.

As an example, the figure below shows the commonly available documentation for the IFC specification, but in this case including the proposed OWL specification. This OWL specification complements the already standardised XSD and EXPRESS specification. The full ifcOWL ontology generated with IfcDoc is identical to the outputs generated with the reference implementations in sections 2.1 and 2.2.



IFC4 - Addendum 1 [Final Standard] © 1996-2015 buildingSMART International Ltd.

|                                               |                                                                                                                                |                                                                                                                          |                                                                                                          |                                                        |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| Cover<br>Contents<br>Foreword<br>Introduction | 1. Scope<br>2. Normative references<br>3. Terms, definitions, and abbreviated terms<br>4. Fundamental concepts and assumptions | 5. Core data schemas<br>6. Shared element data schemas<br>7. Domain specific data schemas<br>8. Resource definition data | A. Computer interpretable listings<br>B. Alphabetical listings<br>C. Inheritance listings<br>D. Diagrams | E. Examples<br>F. Change logs<br>Bibliography<br>Index |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------|

5.1.2.4 IfcPropertySetTemplateTypeEnum  
5.1.2.5 IfcSimplePropertyTemplateTypeEnum  
5.1.2.6 IfcDefinitionSelect  
5.1.2.7 IfcProcessSelect  
5.1.2.8 IfcProductSelect  
5.1.2.9 IfcPropertySetDefinitionSelect  
5.1.2.10 IfcResourceSelect  
5.1.3 Entities  
5.1.3.1 IfcActor  
5.1.3.2 IfcComplexPropertyTemplate  
5.1.3.3 IfcContext  
5.1.3.4 IfcControl  
5.1.3.5 IfcGroup  
5.1.3.6 IfcObject  
5.1.3.7 IfcObjectDefinition  
5.1.3.8 IfcPreDefinedPropertySet  
5.1.3.9 IfcProcess  
5.1.3.10 IfcProduct  
5.1.3.11 IfcProject  
5.1.3.12 IfcProjectLibrary  
5.1.3.13 IfcPropertyDefinition  
5.1.3.14 IfcPropertySet  
5.1.3.15 IfcPropertySetDefinition  
5.1.3.16 IfcPropertySetTemplate  
5.1.3.17 IfcPropertyTemplate  
5.1.3.18 IfcPropertyTemplateDefinition  
5.1.3.19 IfcProxy

**XSD Specification**

```
<xs:simpleType name="IfcPropertySetTemplateTypeEnum">
  <xs:restriction base="xs:string">
    <xs:enumeration value="pset_typedrivenonly"/>
    <xs:enumeration value="pset_typedrivenoverride"/>
    <xs:enumeration value="pset_occurrencedriven"/>
    <xs:enumeration value="pset_performancedriven"/>
    <xs:enumeration value="qto_typedrivenonly"/>
    <xs:enumeration value="qto_typedrivenoverride"/>
    <xs:enumeration value="qto_occurrencedriven"/>
    <xs:enumeration value="notdefined"/>
  </xs:restriction>
</xs:simpleType>
```

**EXPRESS Specification**

**OWL Specification**

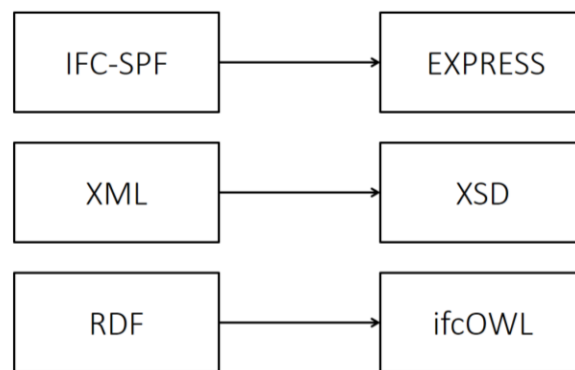
```
ifc:IfcPropertySetTemplateTypeEnum
  rdf:type owl:Class ;
  owl:equivalentClass
  [
    rdf:type owl:Class ;
    owl:oneOf
    (
      ifc:PSET_TYPEDRIVENONLY
      ifc:PSET_TYPEDRIVENVERRIDE
    )
  ]
```



## 4. Validation results

Sections 1 and 2 only document the EXPRESS schemas of IFC (top right in Figure below) and the corresponding ifcOWL ontologies (bottom right in Figure below). As indicated in the Figure below, any IFC building file (IFC-SPF) needs to follow the IFC schema in EXPRESS in order to be valid. Similarly, any IFC building file in RDF needs to follow the ifcOWL ontology in OWL in order to be valid. The same applies to the XML and XSD couple.

To validate the resulting ifcOWL ontology, one needs to be able to assess to what extent any single building model in IFC-SPF (top left in Figure below) can also be expressed as a valid RDF graph (bottom left in Figure below).



In order to have a full ifcOWL BuildingSMART Final Standard, we must validate that the proposed ifcOWL ontology is able to capture the relevant information that was originally available in any of the EXPRESS schemas of IFC and that is needed to properly generate an RDF graph from an IFC-SPF file (cf. Criterion n.3 defined in Section 1). Therefore, we need to be able to prove that round-trip from IFC-SPF to RDF and back is possible (round trip validation). We show in the following sections that:

1. a sample building model in IFC-SPF can also be expressed as an RDF graph
2. the resulting RDF representation of the building model can be made available again as an IFC-SPF file
3. the original IFC-SPF file and the resulting IFC-SPF file (after round-trip) are identical

The above has to be shown for a representative sample set of IFC-SPF building model files. The current document only has the ifcOWL ontology within scope. The representation of the RDF graphs, and the tools used for that purpose (Sections 3.1 and 3.2) are hence out of scope of this standard.

We document the performed round-trip validation in the following subsections.

### 4.1. Tools used for converting IFC-SPF files to RDF files (validation purpose only)

For requirement 1 above (*a sample building model in IFC-SPF can also be expressed as RDF*), two open tools are available and have been used for the round-trip validation documented here.

#### 4.1.1. Open-source IFC-to-RDF JAVA project

Complementary to the JAVA tools documented in Section 2.1 ([ifcParserLib](#) and [EXPRESSToOWL](#)), a third open-source JAVA tool is available for conversion of IFC-SPF files to RDF files:

**URL:** <https://github.com/pipauwel/IFCtoRDF>

**Authors:** Pieter Pauwels, Jyrki Oraskari, Davy Van Deursen, Lewis McGibbney

**Description:** The IFCtoRDF project is a set of reusable Java components that allows to parse IFC-SPF files and convert them to RDF graphs.

By executing this code with any IFC-SPF building model (a .IFC file), a corresponding RDF graph is obtained that is compliant with the corresponding online ifcOWL ontology (a .TTL file).

The code relies on the [ifcParserLib](#) and [EXPRESSToOWL](#) software libraries, as well as the third-party JENA software library for handling RDF graphs and OWL ontologies.

The project is implemented as a MAVEN project.

#### 4.1.2. The IfcDoc tool

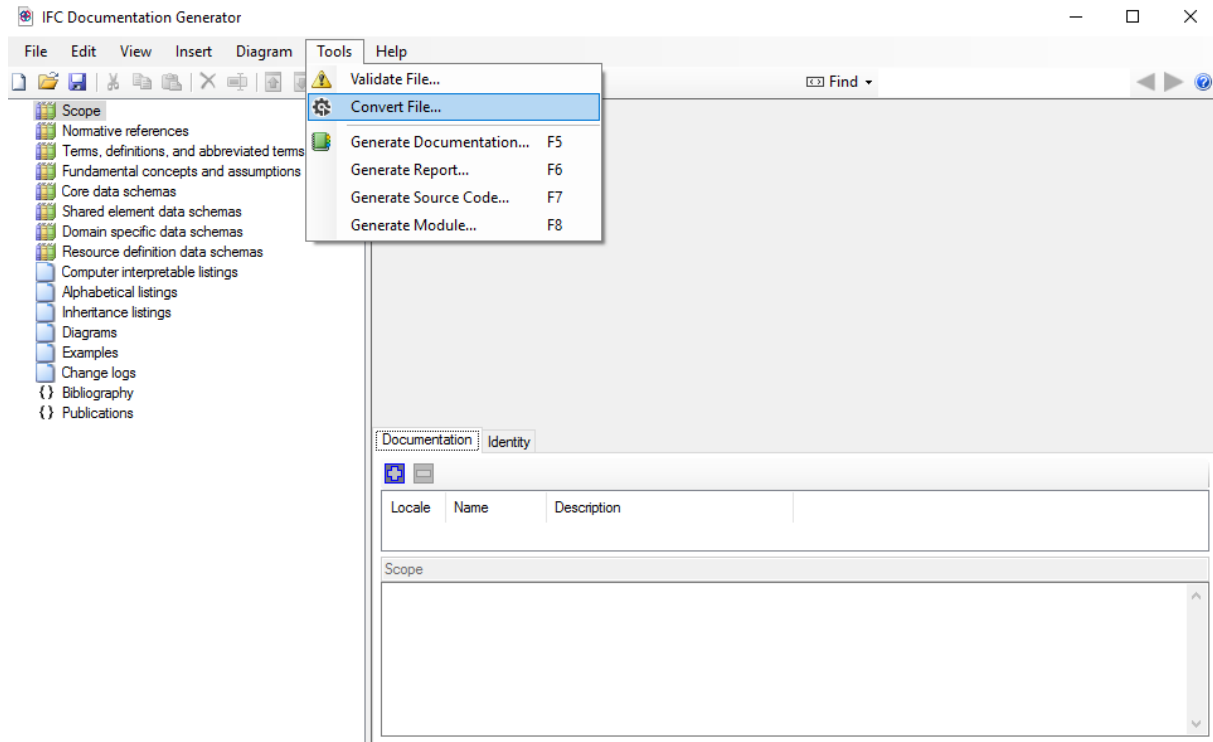
**Authors:** Pieter Pauwels, Walter Terkaj, Tim Chipman

**URL:** <https://github.com/BuildingSMART/IfcDoc>

**Description:**

The IfcDoc tool, available for download and usage through <http://www.buildingsmart-tech.org/specifications/specification-tools/ifcdoc-tool>, contains the required functionality to make any IFC-SPF information that the tool contains also available as RDF graphs.

Furthermore, the IfcDoc tool has been extended to include file-based conversion functionality. This file-based conversion utility currently allows to convert any loaded IFC-SPF building model to a corresponding RDF graph (.TTL file).



## 4.2. Tools used for converting RDF files to IFC-SPF files (validation purpose only)

The conversion of RDF files to IFC-SPF files has been done with a private in-house conversion tool, a tool to convert ifcOWL RDF instances back to IFC STEP files for validation purpose.

**Authors:** Chi Zhang, Jakob Beetz

**Description:** It parses IfcOWL RDF files which are compliant with the corresponding online ifcOWL ontology, and outputs IFC STEP format files.

The tool depends on the open source Apache Jena framework, the dependency and compilation are managed by Maven.

## 4.3. Validation results

Round-trip validation of the ifcOWL ontology has been validated using the IfcDoc tool in Section 3.1.2 (IFC-SPF to RDF) and the private in-house tool in Section 3.2 (RDF to IFC-SPF). The representative sample set used for our validation consists of the IFC-SPF sample files that are available in the IfcDoc tool (IFC4\_ADD1 subset). This includes the 41 files available at Annex E of the IFC4\_ADD1 specification:

<http://www.buildingsmart-tech.org/ifc/IFC4/Add1/html/annex/annex-e.htm>

All files have been converted into RDF graphs and back. The original and resulting files were identical, proving the round-trip validation of the implementations.

Here is the detailed report:

The comparison program is to firstly convert IFC STEP files to ifcOWL RDF files and then convert them back. Original IFC files and roundtrip IFC files are compared with checksum MD5 and SHA1. Some conditions here:

1. As ifcOWL does not cover IFC HEADER data, IFC HEADER data is not considered. Comparison between the original IFC files and round-trip IFC files are only for the DATA part.
2. All the comments, blank lines and spaces that do not affect contents are removed.

The compare program is available in <https://github.com/BenzclyZhang/IFC-RDF-Converter-Validator>. All the tested IFC files, converted IFC RDF files, roundtrip IFC files and detailed compare result in Excel are also there.

The short conclusion is that in <http://www.buildingsmart-tech.org/ifc/IFC4/Add1/html/annex/annex-e.htm> except for one file “wall-elemented-case.ifc”, which is not a valid IFC file (it has an empty reference), all roundtrip files have the same checksum results with original ones.

| IFC files                                  | MD5                              | SHA-1                                     |
|--------------------------------------------|----------------------------------|-------------------------------------------|
| air-terminal-element.ifc                   | ae7444747d41e99b42635c5ad78f4524 | 9bf2ea945433dda80632d34d568f0bc34ad1ed72  |
| air-terminal-element.ifc.ifc               | ae7444747d41e99b42635c5ad78f4524 | 9bf2ea945433dda80632d34d568f0bc34ad1ed72  |
| air-terminal-library-object.ifc            | 2be914cd7d3a7ba6a999b363ea0eff06 | 3dfd1c2cde8ae3ae7e4da1a772413c0652619821  |
| air-terminal-library-object.ifc.ifc        | 2be914cd7d3a7ba6a999b363ea0eff06 | 3dfd1c2cde8ae3ae7e4da1a772413c0652619821  |
| basin-advanced-brep.ifc                    | 8ab76b5b653521a8a2ab22584a545ddf | 5a9bc711caa1a07f572a1f7b43eb4fcd9099899b  |
| basin-advanced-brep.ifc.ifc                | 8ab76b5b653521a8a2ab22584a545ddf | 5a9bc711caa1a07f572a1f7b43eb4fcd9099899b  |
| basin-faceted-brep.ifc                     | ae56221fd8601b02445c41b56c88f2ba | f50838ef62a476baa6050160c31d0081c4fb0fda  |
| basin-faceted-brep.ifc.ifc                 | ae56221fd8601b02445c41b56c88f2ba | f50838ef62a476baa6050160c31d0081c4fb0fda  |
| basin-tessellation.ifc                     | 669fdcc4f165acaca1e9dae71a21a8   | d42ebb1c08c77d11af8bf6fde38bb89513c40af1  |
| basin-tessellation.ifc.ifc                 | 669fdcc4f165acaca1e9dae71a21a8   | d42ebb1c08c77d11af8bf6fde38bb89513c40af1  |
| bath-csg-solid.ifc                         | 3032f041d421c4052eb31eb4e619cb48 | 49ca108ef6f64aa4cb1ffad39ba3b548ff1662b8  |
| bath-csg-solid.ifc.ifc                     | 3032f041d421c4052eb31eb4e619cb48 | 49ca108ef6f64aa4cb1ffad39ba3b548ff1662b8  |
| beam-curved-i-shape-tessel-lated.ifc       | 675dd2555b98178316f7e7f43cd311a6 | ce3b4aa417058c1aa4efeed01a14f44578399c03  |
| beam-curved-i-shape-tessel-lated.ifc.ifc   | 675dd2555b98178316f7e7f43cd311a6 | ce3b4aa417058c1aa4efeed01a14f44578399c03  |
| beam-extruded-solid.ifc                    | 5723901a5658b15a8b7cb31e8156f11e | 32bc0b0c33636d56fd3ea781b778b1c9ff23dd6b  |
| beam-extruded-solid.ifc.ifc                | 5723901a5658b15a8b7cb31e8156f11e | 32bc0b0c33636d56fd3ea781b778b1c9ff23dd6b  |
| beam-revolved-solid.ifc                    | 0609caf9dbdf8ad3ae9cc2510ea7ede  | e00b7c595f266a7f66c325a1cd822fea43bb51e3  |
| beam-revolved-solid.ifc.ifc                | 0609caf9dbdf8ad3ae9cc2510ea7ede  | e00b7c595f266a7f66c325a1cd822fea43bb51e3  |
| beam-standard-case.ifc                     | bf10c87126169332e394107289f4274a | a9e59dbb5751b89ee0a90bcd3f221479d0a28a6f  |
| beam-standard-case.ifc.ifc                 | bf10c87126169332e394107289f4274a | a9e59dbb5751b89ee0a90bcd3f221479d0a28a6f  |
| beam-straight-i-shape-tessel-lated.ifc     | bef62012d3dd32758dc43538f6e16030 | 93f99be3a9df441a9c703c5a52b1f1acbb9337a6  |
| beam-straight-i-shape-tessel-lated.ifc.ifc | bef62012d3dd32758dc43538f6e16030 | 93f99be3a9df441a9c703c5a52b1f1acbb9337a6  |
| beam-varying-cardinal-points.ifc           | a0f27bfee1fbfc1ad41daaaaa1a3d7ab | 4a78eed7889607af9c8cb8a111c61d247a1dca5   |
| beam-varying-cardinal-points.ifc.ifc       | a0f27bfee1fbfc1ad41daaaaa1a3d7ab | 4a78eed7889607af9c8cb8a111c61d247a1dca5   |
| beam-varying-extrusion-paths.ifc           | 1d2091956d20bb83104bd99de293d989 | decc5c4f26075883df2e47b59a9cf2356ceabc68  |
| beam-varying-extrusion-paths.ifc.ifc       | 1d2091956d20bb83104bd99de293d989 | decc5c4f26075883df2e47b59a9cf2356ceabc68  |
| beam-varying-profiles.ifc                  | c0c0d55c5d93cdd11dcf43ad3b4bd7cd | 00b6b05dd9566c8f55c6f3583f9ed2f71c1fb061e |

|                                                     |                                  |                                           |
|-----------------------------------------------------|----------------------------------|-------------------------------------------|
| beam-varying-profiles.ifc.ifc                       | c0c0d55c5d93cdd11dcf43ad3b4bd7cd | 00b6b05dd9566c8f55c6f3583f9ed2f71cfb061e  |
| brep-model.ifc                                      | f92b585fe761205708864e42c00545b8 | d6096bc4a74dd7c03a8162fb6bba44d5d16a88ff  |
| brep-model.ifc.ifc                                  | f92b585fe761205708864e42c00545b8 | d6096bc4a74dd7c03a8162fb6bba44d5d16a88ff  |
| column-extruded-solid.ifc                           | d33557f6db08c16ab235d16fc21e16bb | 29797618276c1c37c0956e313ca2388f17a281c6  |
| column-extruded-solid.ifc.ifc                       | d33557f6db08c16ab235d16fc21e16bb | 29797618276c1c37c0956e313ca2388f17a281c6  |
| column-straight-rectangle-tessel-<br>lation.ifc     | e3d089aff2c45c5f9b6fc11a2e4fab1  | 35f48ca4aa0d8839f4eea05ea42246499d9d3887  |
| column-straight-rectangle-tessel-<br>lation.ifc.ifc | e3d089aff2c45c5f9b6fc11a2e4fab1  | 35f48ca4aa0d8839f4eea05ea42246499d9d3887  |
| construction-scheduling-task.ifc                    | 5d005f77b474caed428c2f84407efab9 | 0fd0da1175e65c7545c9f39367bc41727394505f  |
| construction-scheduling-<br>task.ifc.ifc            | 5d005f77b474caed428c2f84407efab9 | 0fd0da1175e65c7545c9f39367bc41727394505f  |
| csg-primitive.ifc                                   | f57332c4a733087fb0f9a4cae5fb0e20 | 4810087f4eb9f47b02dfd6d853342de2e1fac427  |
| csg-primitive.ifc.ifc                               | f57332c4a733087fb0f9a4cae5fb0e20 | 4810087f4eb9f47b02dfd6d853342de2e1fac427  |
| cube-advanced-brep.ifc                              | 4fafbcaa91ff85c27aa4d32eb66a4a56 | dcd3b3beec2f27cdcd5d90472d24a2e27b7c45351 |
| cube-advanced-brep.ifc.ifc                          | 4fafbcaa91ff85c27aa4d32eb66a4a56 | dcd3b3beec2f27cdcd5d90472d24a2e27b7c45351 |
| extruded-solid.ifc                                  | 5ab6816466cfa92e126b92477f0fe93  | a72ba6863bcdcdce22e446dd2d3af0fde0dc1df27 |
| extruded-solid.ifc.ifc                              | 5ab6816466cfa92e126b92477f0fe93  | a72ba6863bcdcdce22e446dd2d3af0fde0dc1df27 |
| geographic-referencing.ifc                          | 7bc9f99ac001220e10bb374e9d9b1171 | 30b9b41e1fd690a5957df1352d27c58a122e907d  |
| geographic-referencing.ifc.ifc                      | 7bc9f99ac001220e10bb374e9d9b1171 | 30b9b41e1fd690a5957df1352d27c58a122e907d  |
| grid-placement.ifc                                  | 3627b756788b7a881fc4bb2d2c24bbc8 | 7355349dc323d91a0db40807599e020bfc945e63  |
| grid-placement.ifc.ifc                              | 3627b756788b7a881fc4bb2d2c24bbc8 | 7355349dc323d91a0db40807599e020bfc945e63  |
| mapped-shape-with-multiple-<br>items.ifc            | e53fbc4d4381937cc9c7f071886ff6d5 | 43b678ff449668732c6c06ad283a90ae0008ad7e  |
| mapped-shape-with-multiple-<br>items.ifc.ifc        | e53fbc4d4381937cc9c7f071886ff6d5 | 43b678ff449668732c6c06ad283a90ae0008ad7e  |
| mapped-shape-with-transfor-<br>mation.ifc           | cebdf684077b74eca1c14c168d4ab4ac | bf4032d9d26245daf81585258fa1b7effede0418  |
| mapped-shape-with-transfor-<br>mation.ifc.ifc       | cebdf684077b74eca1c14c168d4ab4ac | bf4032d9d26245daf81585258fa1b7effede0418  |
| mapped-shape-without-transfor-<br>mation.ifc        | fcfb3a2721cf0b14350fae4581869cd6 | 0ac624f4edf9ac305a897b229f6121de50776eab  |
| mapped-shape-without-transfor-<br>mation.ifc.ifc    | fcfb3a2721cf0b14350fae4581869cd6 | 0ac624f4edf9ac305a897b229f6121de50776eab  |
| reinforcing-assembly.ifc                            | efc8af6553428fe2e7db08cbbd1ae7e7 | 71722974229e03049571db107b930095f03551ac  |
| reinforcing-assembly.ifc.ifc                        | efc8af6553428fe2e7db08cbbd1ae7e7 | 71722974229e03049571db107b930095f03551ac  |
| reinforcing-stirrup.ifc                             | 5c21e7ad27c3e4ca0a8252732d0d88b6 | 94d47cc72de15f5e2846ab27eb9b4b5df4312c54  |
| reinforcing-stirrup.ifc.ifc                         | 5c21e7ad27c3e4ca0a8252732d0d88b6 | 94d47cc72de15f5e2846ab27eb9b4b5df4312c54  |
| slab-openings.ifc                                   | aef2ae29b9aaf567ec0624ec99e8b515 | 7f321eb89f83dd19fc5d708671bc0136a2d5b956  |
| slab-openings.ifc.ifc                               | aef2ae29b9aaf567ec0624ec99e8b515 | 7f321eb89f83dd19fc5d708671bc0136a2d5b956  |
| slab-standard-case.ifc                              | 86e9ef78e44c6cdcfcd794d4a684e77  | a9d95b521d7f1727dbfcd85417fe8727fc8d0e2b  |
| slab-standard-case.ifc.ifc                          | 86e9ef78e44c6cdcfcd794d4a684e77  | a9d95b521d7f1727dbfcd85417fe8727fc8d0e2b  |
| slab-tessellated-unique-verti-<br>ces.ifc           | af5c0fd84a8bb9d924d264ee503396d2 | 4bbc9403e884eee3803b83eb163fd12db1cd7153  |
| slab-tessellated-unique-verti-<br>ces.ifc.ifc       | af5c0fd84a8bb9d924d264ee503396d2 | 4bbc9403e884eee3803b83eb163fd12db1cd7153  |
| structural-curve-member.ifc                         | 1a7a9e7d134bdba852d13532340b992f | 8b38466352ba453a3315fd04f4265549c6466886  |
| structural-curve-member.ifc.ifc                     | 1a7a9e7d134bdba852d13532340b992f | 8b38466352ba453a3315fd04f4265549c6466886  |
| surface-model.ifc                                   | 3ae2a853674a625162dcedf5e01b5e4a | 50457abdbded62ece90739c1047bd82603e0089d3 |
| surface-model.ifc.ifc                               | 3ae2a853674a625162dcedf5e01b5e4a | 50457abdbded62ece90739c1047bd82603e0089d3 |
| tessellated-item.ifc                                | 17a264058a374470bc888d1f3e00b3c6 | cbd53db7a444ff566129adb5a2321f53aa005f1b  |
| tessellated-item.ifc.ifc                            | 17a264058a374470bc888d1f3e00b3c6 | cbd53db7a444ff566129adb5a2321f53aa005f1b  |
| tessellation-with-blob-texture.ifc                  | e136cf66388ce8ee15d8d2d4473b3827 | 22fe84e660a69de133df0c1c8127ed46492cd99e  |
| tessellation-with-blob-tex-<br>ture.ifc.ifc         | e136cf66388ce8ee15d8d2d4473b3827 | 22fe84e660a69de133df0c1c8127ed46492cd99e  |
| tessellation-with-image-tex-<br>ture.ifc            | 5f78c2224b7fea0e08fc6b5f694b1e6d | dad7aaa9a0efd699eac7da74c7a3b2baa098b82f  |

|                                             |                                   |                                           |
|---------------------------------------------|-----------------------------------|-------------------------------------------|
| tessellation-with-image-texture.ifc.ifc     | 5f78c2224b7fea0e08fc6b5f694b1e6d  | dad7aaa9a0efd699eac7da74c7a3b2baa098b82f  |
| tessellation-with-individual-colors.ifc     | 21529a98907f8bbf0415855456c6853d  | a4e0e5ca41c168dd4bfe4c15d78b03d9906ab3d5  |
| tessellation-with-individual-colors.ifc.ifc | 21529a98907f8bbf0415855456c6853d  | a4e0e5ca41c168dd4bfe4c15d78b03d9906ab3d5  |
| tessellation-with-pixel-texture.ifc         | 8d7daed7a922444f46c47519813b5e6d  | 2c1385707e46dd6c700394015d8d804104a81817  |
| tessellation-with-pixel-texture.ifc.ifc     | 8d7daed7a922444f46c47519813b5e6d  | 2c1385707e46dd6c700394015d8d804104a81817  |
| wall-elemented-case.ifc                     | a35a1fb666b593846a2c2228ee5df2df  | c8fa5c1879c447fa38e31e124666d5b11abd0ed9  |
| wall-elemented-case.ifc.ifc                 | d41d8cd98f00b204e9800998ecf8427e  | da39a3ee5e6b4b0d3255bfe95601890afd80709   |
| wall-standard-case.ifc                      | 9048b2316d00721c45ec99a4110a0422  | 3713dbc1bfdc0df071dccfdb1fe05e7f410c1ec1  |
| wall-standard-case.ifc.ifc                  | 9048b2316d00721c45ec99a4110a0422  | 3713dbc1bfdc0df071dccfdb1fe05e7f410c1ec1  |
| wall-with-opening-and-window.ifc            | 4cff6dcdbd67e248a6b00cd317fb0d35b | 328e55cf77654973fddddd84efe9e60bbcf7af249 |
| wall-with-opening-and-window.ifc.ifc        | 4cff6dcdbd67e248a6b00cd317fb0d35b | 328e55cf77654973fddddd84efe9e60bbcf7af249 |

## 5. Pointer to the full resulting ifcOWL ontology

The latest online version of the ifcOWL ontology is available at:

[http://www.buildingsmart-tech.org/ifcOWL/IFC4\\_ADD1](http://www.buildingsmart-tech.org/ifcOWL/IFC4_ADD1)

The other ifcOWL ontologies are available at:

[http://www.buildingsmart-tech.org/ifcOWL/IFC2X3\\_Final](http://www.buildingsmart-tech.org/ifcOWL/IFC2X3_Final)

[http://www.buildingsmart-tech.org/ifcOWL/IFC2X3\\_TC1](http://www.buildingsmart-tech.org/ifcOWL/IFC2X3_TC1)

<http://www.buildingsmart-tech.org/ifcOWL/IFC4>

[http://www.buildingsmart-tech.org/ifcOWL/IFC4\\_ADD2](http://www.buildingsmart-tech.org/ifcOWL/IFC4_ADD2)

## **6. Pointer to the full HTML documentation of the resulting ifcOWL ontology**

The latest online version of the documentation for IFC4\_ADD1 is available at:

[http://www.buildingsmart-tech.org/ifcOWL/IFC4\\_ADD1](http://www.buildingsmart-tech.org/ifcOWL/IFC4_ADD1)

The other ifcOWL ontologies are available at:

[http://www.buildingsmart-tech.org/ifcOWL/IFC2X3\\_Final](http://www.buildingsmart-tech.org/ifcOWL/IFC2X3_Final)

[http://www.buildingsmart-tech.org/ifcOWL/IFC2X3\\_TC1](http://www.buildingsmart-tech.org/ifcOWL/IFC2X3_TC1)

<http://www.buildingsmart-tech.org/ifcOWL/IFC4>

[http://www.buildingsmart-tech.org/ifcOWL/IFC4\\_ADD2](http://www.buildingsmart-tech.org/ifcOWL/IFC4_ADD2)